

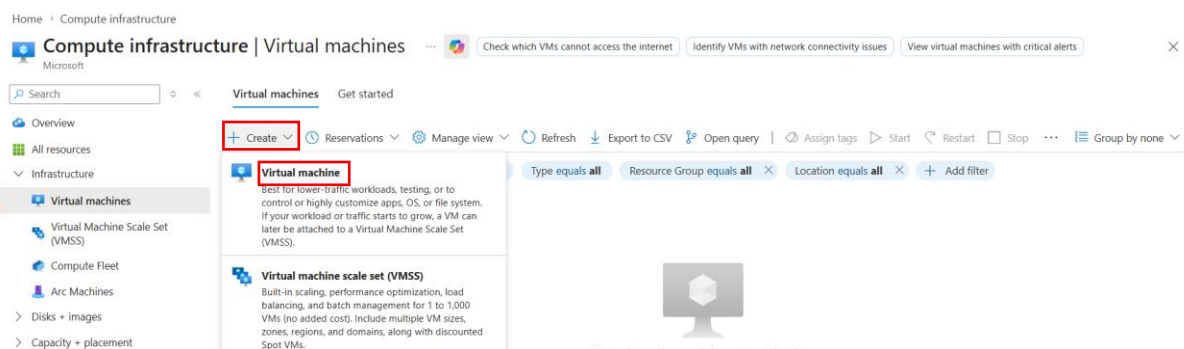
Docker with Azure

To Begin with the Lab

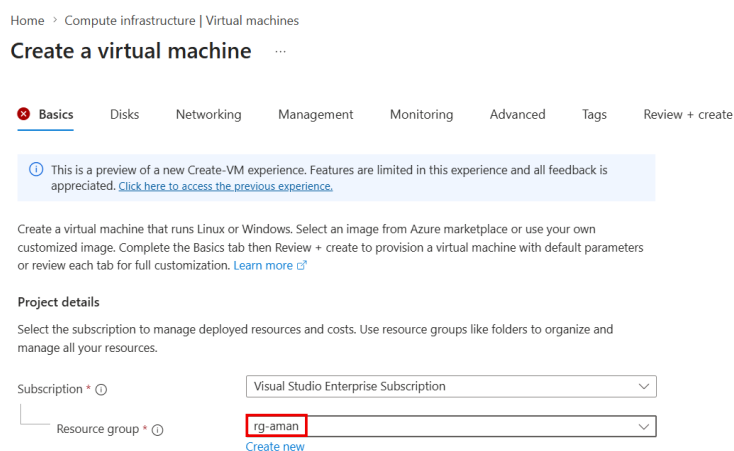
Summary of the Lab

This lab demonstrated how to deploy and manage Docker applications on Microsoft Azure using multiple Azure services. An Ubuntu Linux Virtual Machine was created, Docker and Docker Compose were installed, and MySQL and a PHP application were deployed using Docker Compose. The application database was configured with course data, while course images were migrated from local storage to Azure Blob Storage and accessed through Blob URLs. The Docker images were then pushed to Azure Container Registry (ACR) for centralized image management. Finally, the MySQL container was deployed using Azure Container Instances (ACI), demonstrating how to run containerized applications without managing virtual machines.

- Open the **Azure Portal** and navigate to **Virtual Machines**, then click **Create > Azure Virtual Machine**.



- Select your **Azure Subscription**, create or choose a **Resource Group**, and proceed to the virtual machine configuration page.
- Also, you can create your own.



- Enter a **Virtual Machine Name**, choose the desired **Region**, and set **Availability Options** to **No infrastructure redundancy required**.

Instance details

Virtual machine name * ⓘ

Region * ⓘ
[Deploy to an Azure Extended Zone](#)

Availability options ⓘ

Security type ⓘ

- Under **Image**, select **Ubuntu Server 22.04 LTS (Gen 2)**. If it is not visible, click **See all images**, search for **Ubuntu**, and select the required image.
- Keep the default VM size (**Standard_B1s**) or choose another size if required, then continue.

Image * ⓘ
[See all images](#) | [Configure VM generation](#)

VM architecture * ⓘ ☐ Arm64
☒ x64

Size * ⓘ
[See all sizes](#)

- Select **Password** as the authentication method and provide a valid **Username** and **Password** that meet Azure's password requirements.

Administrator account

Authentication type * ⓘ ☐ SSH public key
☒ Password

Username * ⓘ

Password * ⓘ

Confirm password * ⓘ

- Click **Next: Disks** and keep the default OS disk configuration.
- Click **Next: Networking** and keep the default settings, allowing Azure to create the virtual network, public IP address, network interface, and network security group automatically.
- Continue through the **Management**, **Monitoring**, **Advanced**, and **Tags** tabs without making any changes.
- On the **Review + Create** page, verify the configuration and estimated hourly cost, then click **Create**.
- Wait for the deployment to complete, then click **Go to Resource** to open the newly created virtual machine.
- Open **All Resources** to verify that the deployment has created the **Virtual Machine**, **Public IP Address**, **Virtual Network**, **Network Interface**, **Network Security Group**, **OS Disk**, and other supporting resources successfully.

Home

CreateVirtualMachine-cf-linux-vm | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

Deployment is in progress

Deployment name : CreateVirtualMachine-cf-linux-vm Start time : 7/16/2026, 9:11:54 AM

Subscription : Visual Studio Enterprise Subscription Correlation ID : 96e509a9-f940-4e06-b21a-551ef6c13747

Resource group : rg-aman

Deployment details

Resource	Type	Status	Operation details
✓ vnet-eastusTemplateDeployment17841	Deployment	OK	Operation details
✓ cf-linux-vm-ip	Public IP address	OK	Operation details
✓ cf-linux-vm-nsg	Network security group	OK	Operation details

Next steps

- After the resources are created successfully
- Go to your virtual machine and click connect

Home > CreateVirtualMachine-cf-linux-vm | Overview

cf-linux-vm Virtual machine

Help me copy this VM in any region Manage this VM with Azure CLI

Search

Connect Start Restart Stop Hibernate Capture Delete Refresh Scale Open in mobile Feedback CLI / PS

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Monitor

Resource visualizer

Connect

Networking

Settings

Connect via Bastion

Resource group (move) : rg-aman

Status : Running

Location : East US

Subscription (move) : Visual Studio Enterprise Subscription

Subscription ID : 294539c9-c109-4db8-b785-0043716c0b11

Operating system : Linux (ubuntu 24.04)

Size : Standard DC1ds v3 (1 vcpu, 8 GiB memory)

Primary NIC public IP : 20.120.92.143

1 associated public IPs

Virtual network/subnet : vnet-eastus-1/vnet-eastus-1

DNS name : Not configured

Health state : -

Time created : 7/16/2026, 3:42 AM UTC

Tags (edit) : Add tags

JSON View

- Copy the SSH command

cf-linux-vm | Connect

Virtual machine

Now view, configure, and even save your connection settings — all in one place. Have comments or suggestions for our new Connect experience? Provide feedback

Refresh Reset password or keys Manage JIT Troubleshoot Feedback

Native SSH

Source machine

Source machine OS : Windows

Source IP address : Local IP | 202.83.29.50 Connecting over a VPN?

Destination VM

VM IP address : Public IP | 20.120.92.143

VM port : 22

Connection prerequisites

VM access : Check inbound NSG rules

Check access

SSH command

Execute in your choice of local shell

ssh azureuser@20.120.92.143

Forgot password? Reset password

Edit settings

- Open the command prompt or terminal on the local system and login through the command

```

C:\Users\amanr>ssh azureuser@20.120.92.143
azureuser@20.120.92.143's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1020-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Jul 16 03:48:56 UTC 2026

System load:  0.0          Processes:      113
Usage of /:   5.7% of 28.02GB Users logged in: 0
Memory usage: 3%          IPv4 address for eth0: 172.16.0.4
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Jul 16 03:44:28 2026 from 202.83.29.50
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@cf-linux-vm:~$

```

- Go to docker official [webpage](#) get the step to install the docker

The screenshot shows the Docker Docs website. The left sidebar contains a navigation menu with categories like 'AI AND AGENTS', 'APPLICATION DEVELOPMENT', and 'Storage'. The main content area is titled 'Install using the apt repository' and includes instructions for setting up Docker's apt repository. A code block shows the commands to add Docker's official GPG key and the repository to Apt sources. The right sidebar contains a 'Table of contents' with links to prerequisites, firewall limitations, OS requirements, and installation methods.

dockerdocs Get started Guides **Manuals** Reference Gordon Search

AI AND AGENTS
Overview
Docker Sandboxes
MCP Catalog and Toolkit
Model Runner
Gordon
AI and Docker Compose
Docker Agent

APPLICATION DEVELOPMENT
Docker Desktop
Docker Engine
Install
Ubuntu
Debian
RHEL
Fedora
Raspberry Pi OS (32-bit / armhf)
CentOS
Binaries
Post-installation steps
Storage
Networking

Apache License, Version 2.0. See [LICENSE](#) for the full license.

Install using the apt repository

Before you install Docker Engine for the first time on a new host machine, you need to set up the Docker apt repository. Afterward, you can install and update Docker from the repository.

1. Set up Docker's apt repository.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/g
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

[Edit this page](#)
[Request changes](#)

Table of contents

- Prerequisites
- Firewall limitations
- OS requirements
- Uninstall old versions
- Installation methods
 - Install using the apt repository
 - Upgrade Docker Engine
 - Install from a package
 - Upgrade Docker Engine
 - Install using the convenience script
 - Install pre-releases
 - Upgrade Docker after using the convenience script
 - Uninstall Docker Engine
- Next steps

[Give feedback](#)

- After running all the command docker is installing

```

azureuser@cf-linux-vm:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras pigz
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin pigz
0 upgraded, 7 newly installed, 0 to remove and 8 not upgraded.
Need to get 99.4 MB of archives.
After this operation, 380 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.6-1~ubuntu.24.04~noble [23.6 MB]
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:29.6.1-1~ubuntu.24.04~noble [16.9 MB]
Get:4 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:29.6.1-1~ubuntu.24.04~noble [23.3 MB]
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin amd64 0.35.8-1~ubuntu.24.04~noble [17.2 MB]
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-rootless-extras amd64 5:29.6.1-1~ubuntu.24.04~noble [10.2 MB]
Get:7 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-compose-plugin amd64 5.3.1-1~ubuntu.24.04~noble [8100 kB]
Fetched 99.4 MB in 1s (143 MB/s)
Selecting previously unselected package containerd.io.
(Reading database ... 69373 files and directories currently installed.)
Preparing to unpack .../0-containerd.io_2.2.6-1~ubuntu.24.04~noble_amd64.deb ...
Unpacking containerd.io (2.2.6-1~ubuntu.24.04~noble) ...
Selecting previously unselected package docker-ce-cli.
Preparing to unpack .../1-docker-ce-cli_5%3a29.6.1-1~ubuntu.24.04~noble_amd64.deb ...
Unpacking docker-ce-cli (5:29.6.1-1~ubuntu.24.04~noble) ...
Selecting previously unselected package docker-ce.
Preparing to unpack .../2-docker-ce_5%3a29.6.1-1~ubuntu.24.04~noble_amd64.deb ...
Unpacking docker-ce (5:29.6.1-1~ubuntu.24.04~noble) ...
Progress: [ 17%] [#####.....]

```

- Let's check the docker version

```

azureuser@cf-linux-vm:~$ sudo docker --version
Docker version 29.6.1, build 8900f1d
azureuser@cf-linux-vm:~$

```

Running a MySQL Docker Container on the Azure Linux VM

- Verify that no Docker images are available on the Linux VM:

```
sudo docker images
```

- Run a MySQL container by pulling the image from Docker Hub and publishing port **3306**:

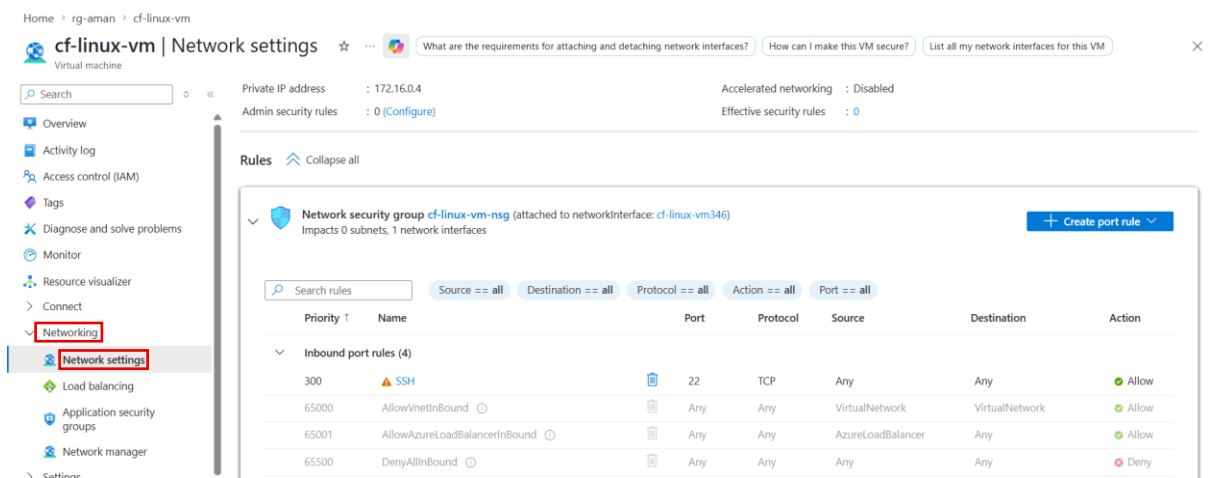
```
sudo docker run -d -p 3306:3306 --name mysql-server -e
MYSQL_ROOT_PASSWORD=1234 mysql
```

- Verify that the MySQL container is running:

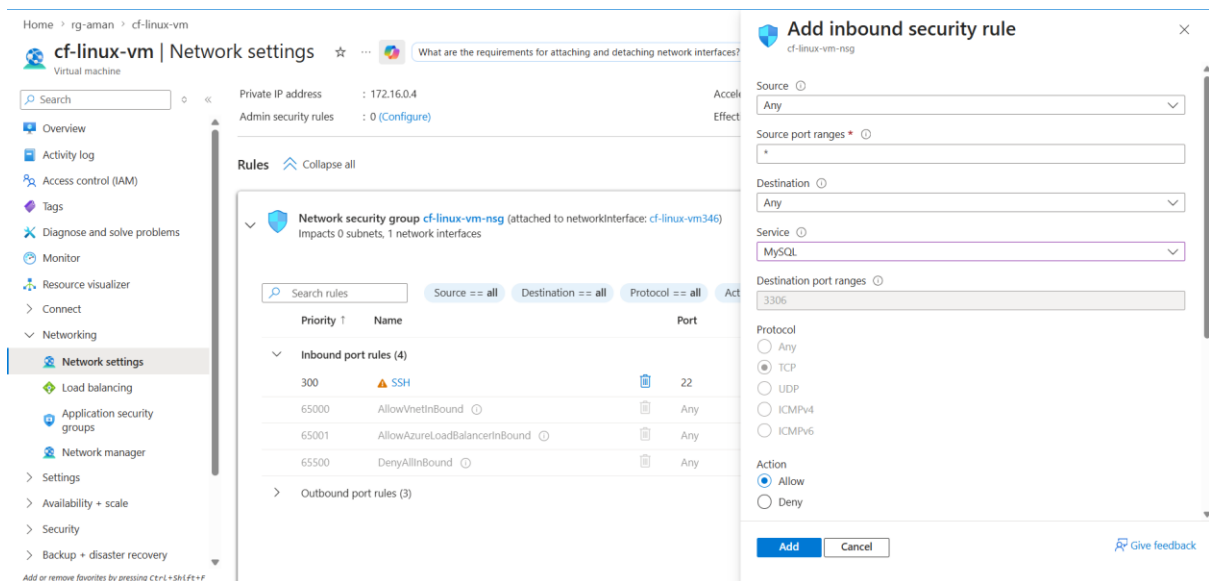
```
sudo docker ps
```

```
azureuser@cf-linux-vm:~$ sudo docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
azureuser@cf-linux-vm:~$ sudo docker run -d -p 3306:3306 --name mysql-server -e MYSQL_ROOT_PASSWORD=1234 mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
446597dc68d2: Pull complete
ecf47111addb: Pull complete
d4c51daaa784: Pull complete
ad18077cf381: Pull complete
9f5b662d2d4d: Pull complete
890dd41ed99a: Pull complete
1257e53bba2a: Pull complete
6b6fad81c717: Pull complete
aef22d41055: Pull complete
804eb080c2a10: Pull complete
62f7e05590ba: Download complete
54e5e996b461: Download complete
Digest: sha256:ae269281abffe401d65f04cb54d45a069a495b8174b9c0a815e502fed7fa0370
Status: Downloaded newer image for mysql:latest
1ea6c3fb8a78c3c41082c988f632795e3161dce18c40f017d05ba1046b0dffa2
azureuser@cf-linux-vm:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
1ea6c3fb8a78 mysql "docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp mysql-se
rver
azureuser@cf-linux-vm:~$
```

- In the **Azure Portal**, open the **Virtual Machine**, navigate to **Networking**, and click **Add inbound port rule**.

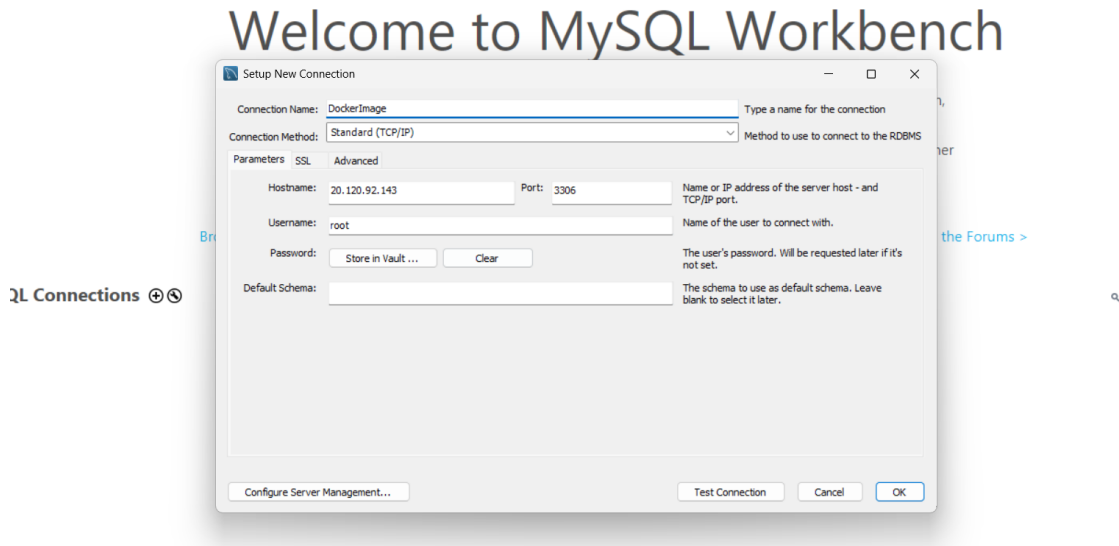


- Select the **MySQL (TCP 3306)** service (or manually allow **TCP port 3306**) and create the inbound rule.

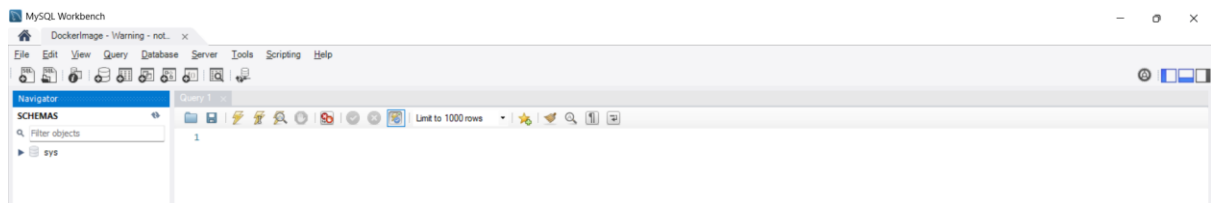


- Copy the **Public IP Address** of the virtual machine from the **Overview** page.

- Open **MySQL Workbench** (or any MySQL client), create/edit a connection, and use the VM's **Public IP Address** as the hostname.
- Enter the **root** username and the password specified while creating the container, then connect to verify the MySQL server is accessible.



- Confirm that the MySQL database is running successfully on the Docker container hosted inside the Azure Linux Virtual Machine.



Running Docker Compose on the Azure Linux VM

- Navigate to the application directory containing the Dockerfile, docker-compose.yaml, PHP application, and Bootstrap files.
- Verify that no containers are currently running:

```
sudo docker ps -a
```

- Create a new Docker Compose file on the VM:

```
cat > docker-compose.yaml
```

- Create or copy the docker-compose.yaml file to the Linux VM and verify its contents.
- Verify that the file has been created:

```
ls
```

View the contents of the Docker Compose file:

```
cat docker-compose.yaml
```

- code

```
services:
  webapp:
    image: phpapp:v3
    build: .
    container_name: phpapp-instance
    depends_on:
      - db
    networks:
      - app-network
    ports:
      - "80:80"

  db:
    image: mysql:latest
    container_name: mysql-instance
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: sqlset1010
      MYSQL_DATABASE: appdb
      MYSQL_USER: appusr
      MYSQL_PASSWORD: appusr@1090
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    networks:
      - app-network
```


networks:

app-network:

driver: bridge

ipam:

config:

- subnet: 172.28.0.0/16

volumes:

mysql-data:

- Build and deploy the application using Docker Compose.

```
sudo docker compose up -d --build
```

- Verify that the Docker image, custom network, volume, and both containers are created successfully.

```
azureuser@cf-linux-vm:~/application$ sudo docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
mysql:latest ae269281abff 1.28GB 288MB U
phpapp:v3 7283963a58dd 716MB 177MB U
azureuser@cf-linux-vm:~/application$ sudo docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
08ca4f9f276f phpapp:v3 "docker-php-entrypoi..." 13 minutes ago Up 3 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp phpapp-inst
e681194beafd mysql:latest "docker-entrypoint.s..." 13 minutes ago Up 13 minutes 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp mysql-insta
nce
azureuser@cf-linux-vm:~/application$ sudo docker volume ls
DRIVER VOLUME NAME
local 1ed12f53bba52e4c18b8a6c3198bc08ff137f7086a8fc0d2ff104dfe2a61501
local 0759e3f3177d854d7798ae24c008cab1426bbda5998c14275e71c80ec19f7b
local 900594e1a5259de29ffde2f92c31919badd69eeb2ce15d360caad67fffd5f4f5
local a95a30a5c899867c343802054456b9cfe3afe12f0f68142761b1e7de6cce8c8
local a4350978716e54c84030a60ce99d947ec0493c346a7566584c0d14b5209de1b5
local application_mysql-data
azureuser@cf-linux-vm:~/application$ sudo docker network ls
NETWORK ID NAME DRIVER SCOPE
123a1f83b222 application_app-network bridge local
b57cb59b5c59 bridge bridge local
013a0c589a59 host host local
668b73b02e0 none null local
azureuser@cf-linux-vm:~/application$
```

- Enter the MySQL container

```
sudo docker exec -it mysql-instance mysql -uroot -psqlset1010
```

- Before creating table, check databases

```
show databases;
```

- check for appdb
- Create the table, Inside MySQL, execute:

```
USE appdb;
```

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    ExamImage VARCHAR(255),
    CourseName VARCHAR(100),
    rating DECIMAL(2,1)
);
```

```
INSERT INTO Course VALUES
```

```
(101,'https://via.placeholder.com/400x100','Docker Fundamentals',4.8),
(102,'https://via.placeholder.com/400x100','Docker Compose',4.7),
(103,'https://via.placeholder.com/400x100','Azure for Containers',4.9),
(104,'https://via.placeholder.com/400x100','Kubernetes Basics',4.6);
```

```
GRANT ALL PRIVILEGES ON appdb.* TO 'appusr'@'%';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

- In the Azure Portal, open the VM's Networking settings and add an HTTP (TCP Port 80) inbound rule.

Inbound port rules (6)							
300	SSH		22	TCP	Any	Any	Allow
310	database_connection		3306	TCP	Any	Any	Allow
330	port		80	TCP	Any	Any	Allow
65000	AllowVnetInBound		Any	Any	VirtualNetwork	VirtualNetwork	Allow

- Copy the VM's Public IP Address and open it in a browser to verify that the PHP application is running successfully.

Courses

This is a list of Courses

Course ID	Course Image	Course Name	Rating
101		Docker Fundamentals	4.8
102		Docker Compose	4.7
103		Azure for Containers	4.9
104		Kubernetes Basics	4.6

Storing Images in Azure Blob Storage

- Navigate to **Storage Account**
- Create a new **Azure Storage Account** using the same subscription and resource group as the Linux Virtual Machine.

Home > Storage center

Storage center | Blob Storage ... Analyze alerts across storage accounts Help me identify cost-saving opportunities in storage accounts List storage accounts with potential vulnerabilities

My Directory (behalriteshg@mail.onmicrosoft.com)

Search Overview All storage resources Object storage File storage Block storage Data management

Summary Resources

+ Create Restore Manage view Refresh Export to CSV Open query Assign tags Delete Add to service group Group by none

Filter for any field... Subscription equals all Resource Group equals all Location equals all Add filter

Name ↑	Type	Kind	Resource Group	Location	Subscription	Defender for stor...
funcdeveus01groupb92d	Storage account	StorageV2	func-dev-eus-01.g...	East US	Visual Studio Enter...	Not protected

- Provide a unique name for the storage account, select the desired region, choose **Locally Redundant Storage (LRS)**, and complete the deployment.

Home > Storage center | Blob Storage

Create a storage account ...

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Visual Studio Enterprise Subscription

Resource group * rg-aman
[Create new](#)

Instance details

Storage account name * sqpics

Region * (US) East US
[Deploy to an Azure Extended Zone](#)

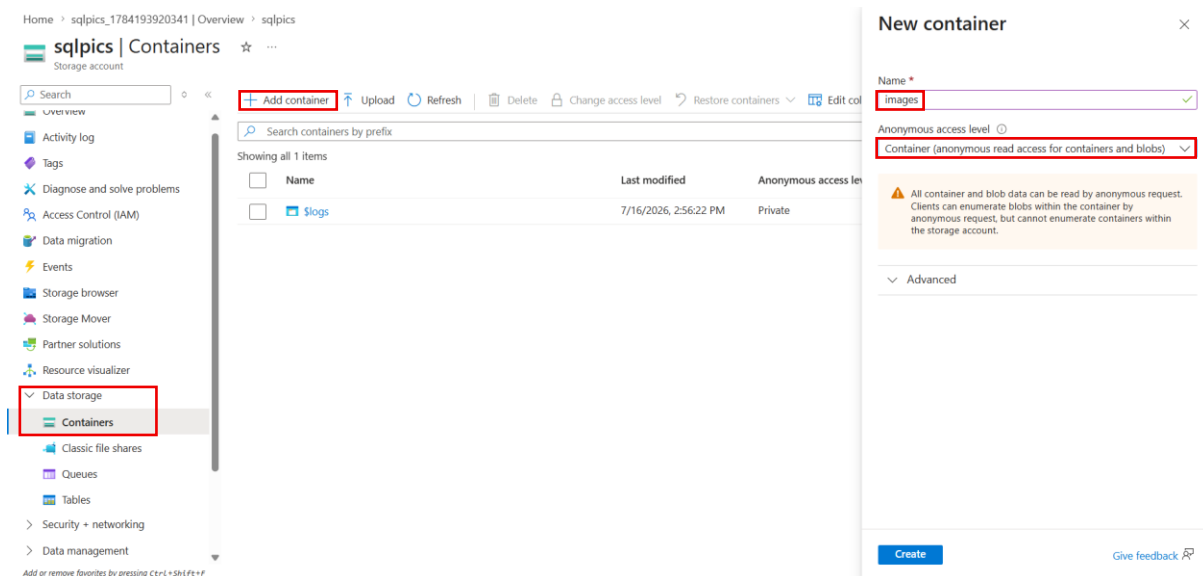
Primary service * **Azure Blob Storage or Azure Data Lake Storage**

Performance * Standard: Recommended for most scenarios (general-purpose v2 account)
Premium: Recommended for scenarios that require low latency.

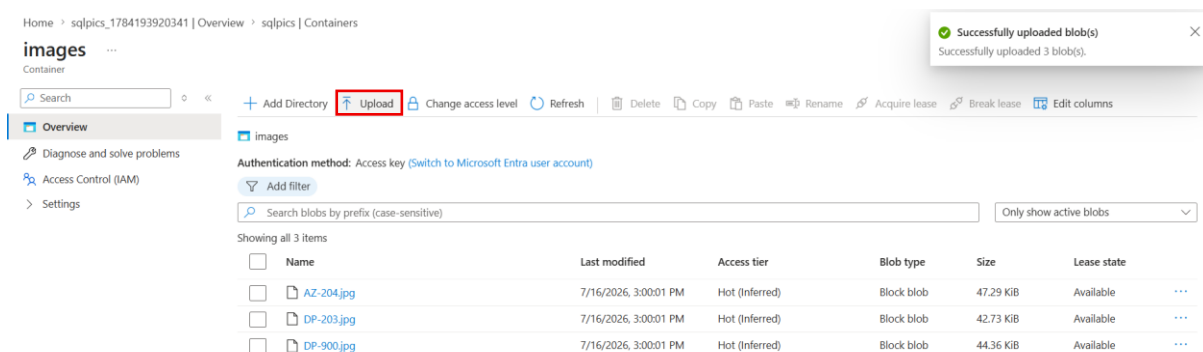
Redundancy * **Locally redundant storage (LRS)**

- Open the newly created Storage Account after the deployment is completed.

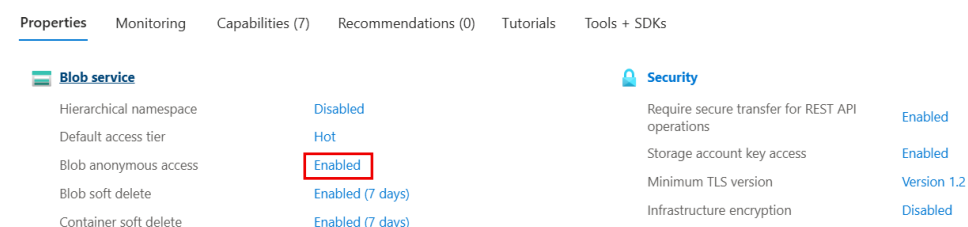
- Navigate to **Containers** under the Data storage and create a new container named **images**.



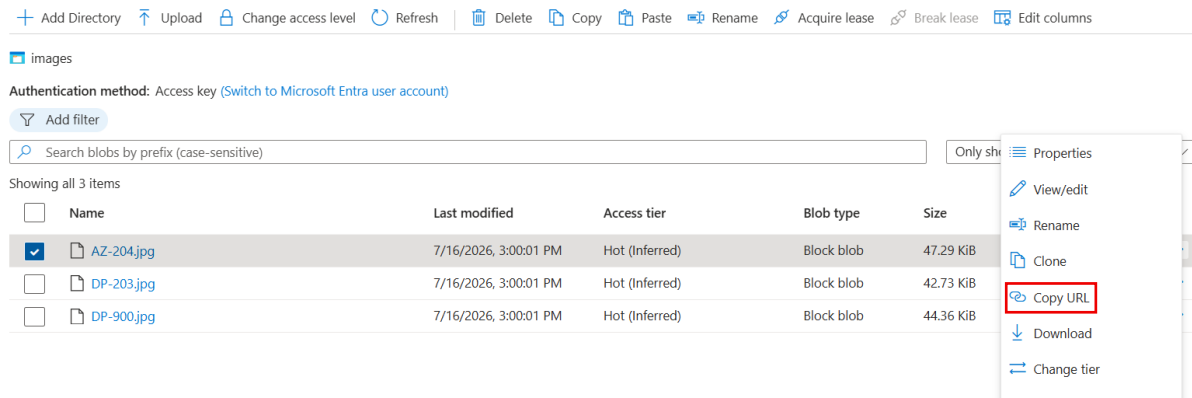
- Open the **images** container and upload the required course image files from the local machine.
- Verify that all image files have been uploaded successfully to the Blob container.



- Go to the **Configuration** section of the Storage Account and enable **Allow Blob Anonymous Access**.
- Save the configuration changes and refresh the Storage Account page.



- Open the **images** container, select **Change Access Level**, and set the access level to **Blob (Anonymous read access for blobs only)**.
- Save the updated access level for the container.
- Open any uploaded image and copy its generated Blob URL.



Deploying the Updated Application with Azure Blob Storage Images

- Connect to the MySQL database and verify that the **Course** table contains the Course ID, Exam Image, Course Name, and Rating columns.

```
sudo docker exec -it mysql-instance mysql -uroot -psqlset1010
```

- Copy the Azure Blob Storage URLs for the uploaded course images from the Azure Storage Account.
- Execute SQL UPDATE statements to replace the existing image paths with the Azure Blob Storage URLs stored in the **ExamImage** column.

```
UPDATE Course
```

```
SET ExamImage='https://sqlpics.blob.core.windows.net/images/AZ-204.jpg'
```

```
WHERE CourseID=101;
```

```
UPDATE Course
```

```
SET ExamImage='https://sqlpics.blob.core.windows.net/images/DP-203.jpg'
```

```
WHERE CourseID=102;
```

```
UPDATE Course
```

```
SET ExamImage='https://sqlpics.blob.core.windows.net/images/DP-900.jpg'
```

```
WHERE CourseID=103;
```

```
UPDATE Course
```

```
SET ExamImage='https://sqlpics.blob.core.windows.net/images/AZ-204.jpg'
```

```
WHERE CourseID=104;
```

- Verify that the **ExamImage** column now contains the complete Blob Storage URLs for all course images.

```
SELECT * FROM Course;
```

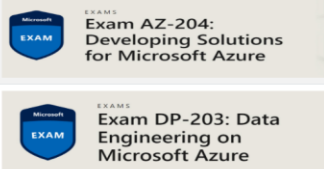
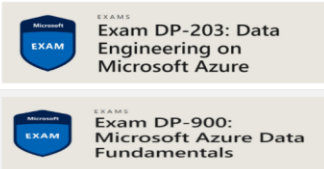
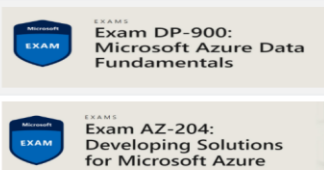
```
mysql> SELECT * FROM Course;
+-----+-----+-----+-----+
| CourseID | ExamImage | CourseName | rating |
+-----+-----+-----+-----+
| 101 | https://via.placeholder.com/400x100 | Docker Fundamentals | 4.8 |
| 102 | https://via.placeholder.com/400x100 | Docker Compose | 4.7 |
| 103 | https://via.placeholder.com/400x100 | Azure for Containers | 4.9 |
| 104 | https://via.placeholder.com/400x100 | Kubernetes Basics | 4.6 |
+-----+-----+-----+-----+
4 rows in set (0.000 sec)

mysql> |
```

- Open the PHP application using the Azure Linux VM Public IP address.
- <http://<Public-IP>/index.php>
- Refresh the application and verify that all course images are successfully loaded from Azure Blob Storage.
- Confirm that the PHP application retrieves the course details from MySQL while displaying the images directly from Azure Blob Storage.

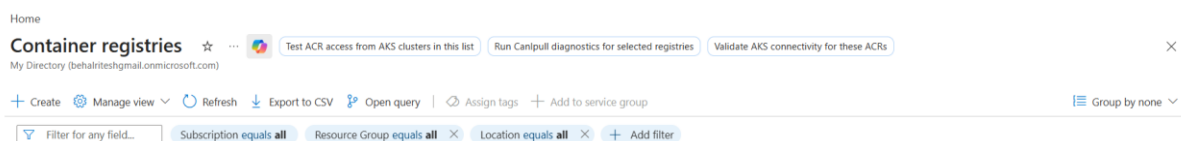
Courses

This is a list of Courses

Course ID	Course Image	Course Name	Rating
101		Docker Fundamentals	4.8
102		Docker Compose	4.7
103		Azure for Containers	4.9
104		Kubernetes Basics	4.6

Pushing Docker Images to Azure Container Registry (ACR)

- Create a new **Azure Container Registry (ACR)** resource in the existing Resource Group and choose the desired Azure region.



- Provide a unique registry name, keep the default pricing tier (**Basic**), and complete the deployment.

Home > Container registries

Create container registry

Basics Networking Encryption Tags Review + create

Azure Container Registry allows you to build, store, and manage container images and artifacts in a private registry for all types of container deployments. Use Azure container registries with your existing container development and deployment pipelines. Use Azure Container Registry Tasks to build container images in Azure on-demand, or automate builds triggered by source code updates, updates to a container's base image, or timers. [Learn more](#)

Project details

Subscription * Visual Studio Enterprise Subscription

Resource group * rg-aman [Create new](#)

Instance details

Registry name * cf01ACR ✓
azurecr.io

Location * East US

Domain name label scope * Unsecure

Registry domain name cf01acr.azurecr.io

Pricing plan * Basic

Role assignment permissions mode ☐ RBAC Registry + ABAC Repository Permissions ☒ RBAC Registry Permissions

- Open the newly created Azure Container Registry from the Azure Portal.
- Navigate to **Repositories** and verify that no repositories are present initially.

Home > Microsoft.ContainerRegistry | Overview > cf01ACR

cf01ACR | Repositories

Container registry

Search Refresh Manage Deleted Repositories

Search to filter repositories ...

Repositories ↑↓ Cache Rule

No result

Overview Activity log Access control (IAM) Tags Diagnose and solve problems Quick start Resource visualizer Events Settings Services **Repositories** Webhooks

- Go to **Access Keys**, enable the **Admin User**, and note the **Login Server**, **Username**, and **Password**.

Home > Microsoft.ContainerRegistry | Overview > cf01ACR

cf01ACR | Access keys

Container registry

Search Registry name cf01ACR

Login server cf01acr.azurecr.io

Admin user ☒

Username cf01ACR

Name	Password	Regenerate
password		Show
password2		Show

Overview Activity log Access control (IAM) Tags Diagnose and solve problems Quick start Resource visualizer Events Settings **Access keys** Encryption

- Connect to the Azure Linux Virtual Machine and stop the running Docker Compose application.

```
sudo docker compose down
```

- Log out of Docker Hub.

```
docker logout
```

- Log in to the Azure Container Registry using the Login Server, Username, and Password.

```
docker login <login-server>
```

```
azureuser@cf-linux-vm:~$ sudo docker images
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
mysql:latest   ae269281abff      1.28GB      288MB
phpapp:v3      fc552b268ff0      725MB       178MB
azureuser@cf-linux-vm:~$ docker logout
Removing login credentials for https://index.docker.io/v1/
azureuser@cf-linux-vm:~$ sudo docker login cf01acr.azurecr.io
Username: cf01ACR
Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
azureuser@cf-linux-vm:~$ |
```

- Tag the local PHP application image with the Azure Container Registry login server.

```
docker tag phpapp:v3 <login-server>/phpapp:v3
```

- Push the MySQL image to Azure Container Registry.

```
docker push <login-server>/mysql:latest
```

- Then tag the images:

```
sudo docker tag phpapp:v3 cf01acr.azurecr.io/phpapp:v3
```

```
sudo docker tag mysql:latest cf01acr.azurecr.io/mysql:latest
```

- Push them:

```
sudo docker push cf01acr.azurecr.io/phpapp:v3
```

```
sudo docker push cf01acr.azurecr.io/mysql:latest
```

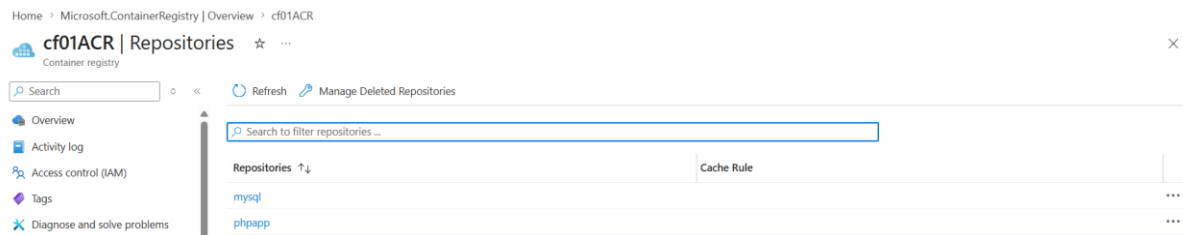
- This is the easiest method because the images are already on the VM.


```

azureuser@cf-linux-vm:~$ sudo docker tag phpapp:v3 cf01acr.azurecr.io/phpapp:v3
azureuser@cf-linux-vm:~$ sudo docker tag mysql:latest cf01acr.azurecr.io/mysql:latest
azureuser@cf-linux-vm:~$ sudo docker push cf01acr.azurecr.io/phpapp:v3
The push refers to repository [cf01acr.azurecr.io/phpapp]
e7daf0503344: Pushed
092a953801a3: Pushed
a5587406c34b: Pushed
b00eb207193a: Pushed
d3b60191b176: Pushed
062e450697fa: Pushed
3893442b8746: Pushed
180d13d999fe1: Pushed
8b3caee61af: Pushed
8b192518ec74: Pushed
91e54e4d2883: Pushed
17ff64b82f58: Pushed
d4d82b592474: Pushed
b1ff61be1778: Pushed
5d1bd08eeb06: Pushed
4f4fb700ef54: Pushed
1468762af3b3: Pushed
f916e83bd43c: Pushed
2e0c3120a4ef: Pushed
v3: digest: sha256:fc552b268ff08bb6efc5ad5d26e6569e3c692c8895156721f555bbb6a4776de0 size: 856
azureuser@cf-linux-vm:~$ sudo docker push cf01acr.azurecr.io/mysql:latest
The push refers to repository [cf01acr.azurecr.io/mysql]
044eb080c2a10: Pushed
ecf47111adb: Pushed
d4c51daaa784: Pushed
9f5b662d2d4d: Pushed
890dd41ed99a: Pushed
aeff22d41055: Pushed
446597dc68d2: Pushed
ad18077cf381: Pushed
1257e53bba2a: Pushed
6b6fad81c717: Pushed
latest: digest: sha256:91e42ae952f37b724f945c734cf067c65aa66bf15d4992a7f7c174918d15fc77 size: 2856

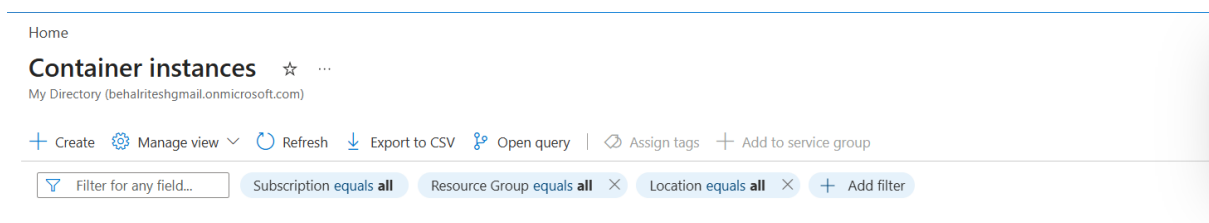
```

- Refresh the **Repositories** page in Azure Container Registry and verify that both the **phpapp** and **mysql** repositories are available.



Deploying a Container using Azure Container Instances (ACI)

- In the Azure Portal, create a new **Azure Container Instance** resource.



- Select the existing **Resource Group**, provide a unique **Container Name**, and choose the desired Azure region.
- Under **Image Source**, select **Azure Container Registry (ACR)** and choose the previously pushed **MySQL** image (mysql:latest in your case).
- Configure the container with **1 vCPU** and **1.5 GB Memory**, leaving the remaining settings at their default values.

Home > Container instances

Create container instance

Container name * ⓘ sqlinstance0012 ✓

Region * ⓘ (US) East US ✓

Availability zones (Preview) ⓘ None ✓

SKU Standard ✓

Image source * ⓘ ☐ Quickstart images ☒ Azure Container Registry ☐ Other registry

Run with Azure Spot discount ⓘ ☐
 ⓘ Spot containers are not available in the selected region. [Learn more](#) ⓘ

Registry ⓘ cf01ACR ✓
 ⓘ If you do not see your Azure Container Registry, ensure you have been assigned the Reader Role for the Azure Container Registry or select an Azure Container Registry in a different subscription. [Learn more](#) ⓘ

Image * ⓘ mysql ✓

Image tag * ⓘ latest ✓

OS type Linux

Size * ⓘ 1 vcpu, 1.5 GiB memory, 0 gpus

- In the **Networking** tab, select **Public** for network access and expose **Port 3306** to allow MySQL connections.

Basics **Networking** Monitoring Advanced Tags Review + create

Choose between three networking options for your container instance:

- **'Public'** will create a public IP address for your container instance.
- **'Private'** will allow you to choose a new or existing virtual network for your container instance.
- **'None'** will not create either a public IP or virtual network. You will still be able to access your container logs using the command line.

Networking type ☒ Public ☐ Private ☐ None

DNS name label ⓘ

DNS name label scope reuse ⓘ Any reuse (unsecure) ✓

Ports ⓘ

Ports	Ports protocol
3306 ✓	TCP ✓

- In advance configure the environments

Home > Container instances

Create container instance

Basics Networking Monitoring **Advanced** Tags Review + create

Configure additional container properties and variables.

Restart policy ⓘ On failure ✓

Environment variables

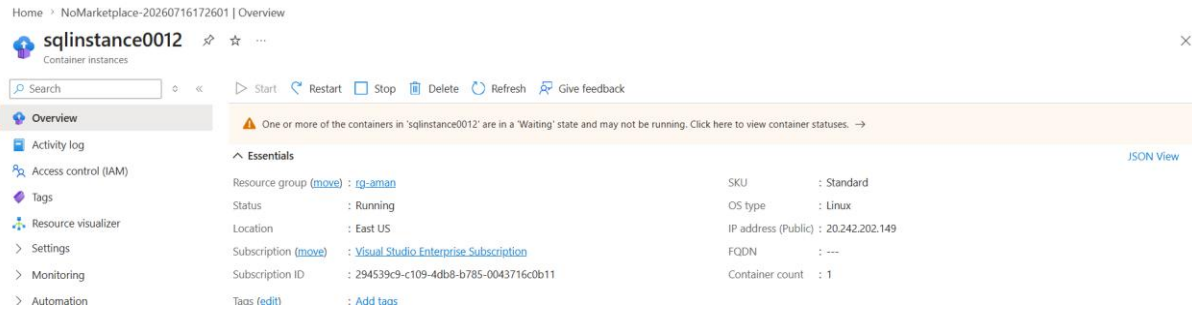
Mark as secure	Key	Value
Yes	MYSQL_ROOT_PASSWORD	***** ✓
Yes	MYSQL_DATABASE	***** ✓
Yes	MYSQL_USER	***** ✓
Yes ✓	MYSQL_PASSWORD ✓	***** ✓
No		

Command override ⓘ [] ✓
 Example: ["/bin/bash", "-c", "echo hello; sleep 100000"]

Key management ⓘ ☒ Microsoft-managed keys (MMK) ☐ Customer-managed keys (CMK)

- Complete the remaining configuration using the default settings and click **Review + Create**, followed by **Create** to deploy the container instance.

- After deployment, open the **Container Instance** resource and verify that the container is in the **Running** state.



- Open the **Containers** section to view the container status, properties, events, and logs generated by the running container.
- Copy the **Public IP Address** assigned to the Azure Container Instance.
- Open **MySQL Workbench**, edit an existing MySQL connection (or create a new one), replace the hostname with the **Public IP Address** of the container instance, and connect using the configured MySQL credentials.

